

Integrantes:

- Jorge Figueredo
- Rodrigo Maidana
- Pedro Kaslauskas
- Daniel Ferreira

1. ¿Cuál es la diferencia entre un Pod y un Deployment?

Un Pod es la unidad mínima de ejecución en Kubernetes: contiene uno o más contenedores que comparten red y almacenamiento. Sin embargo, un Pod por sí solo es efímero — si falla o se elimina, no se recupera automáticamente.

Un Deployment es un controlador que gestiona el ciclo de vida de los pods. Define cuántas réplicas deben estar corriendo en todo momento y se encarga de recrear pods si fallan, actualizar versiones de forma controlada y escalar horizontalmente. En el proyecto, cada componente (postgres, backend, frontend) tiene su propio Deployment que garantiza que siempre exista al menos una instancia corriendo.

2. ¿Por qué Kubernetes recrea un pod automáticamente?

Kubernetes implementa el principio de estado deseado (desired state). Al definir replicas: 1 en un Deployment, el sistema registra que siempre debe haber 1 pod corriendo. Un componente llamado Controller Manager monitorea continuamente el estado real del clúster y lo compara con el estado deseado. Si detecta que un pod fue eliminado o falló, automáticamente crea uno nuevo para reconciliar ambos estados.

Esto se verificó en el proyecto ejecutando:
`kubectrl delete pod <nombre>`

Kubernetes inmediatamente comenzó a crear un pod de reemplazo, visible con:
`kubectrl get pods -w`

3. ¿Qué problema resuelve un Service?

Los pods en Kubernetes tienen direcciones IP dinámicas que cambian cada vez que se recrean. Sin un Service, sería imposible que el backend siempre encuentre a PostgreSQL, ya que la IP puede haber cambiado.

Un Service provee una IP y nombre DNS estables que actúan como punto de entrada fijo hacia un conjunto de pods. En el proyecto:

- El postgres-service (ClusterIP) permite que el backend siempre alcance la base de datos usando el hostname postgres-service, sin importar cuántas veces se haya recreado el pod.
- Los services de tipo NodePort del backend y frontend exponen la aplicación fuera del clúster en puertos fijos (30300 y 30517), permitiendo el acceso desde el browser.

4. ¿Cuál es la diferencia entre ConfigMap y Secret?

Ambos son mecanismos para externalizar la configuración de los contenedores, pero difieren en su propósito y nivel de protección:

- ConfigMap: almacena datos de configuración no sensibles en texto plano, como DB_HOST, DB_PORT, PORT y CORS_ORIGIN. Es legible directamente con:
`kubectl get configmap`

- Secret: almacena información sensible como contraseñas y credenciales. Los valores se codifican en base64 y Kubernetes ofrece mecanismos adicionales de protección (acceso restringido por RBAC, posibilidad de cifrado en reposo). En el proyecto, las credenciales de PostgreSQL (POSTGRES_USER, POSTGRES_PASSWORD) se almacenan en un Secret que tanto el Deployment de postgres como el del backend consumen mediante referencias directas (secretKeyRef).

La separación evita que credenciales sensibles queden expuestas junto a la configuración general.

5. ¿Qué ocurre si un pod falla constantemente?

Kubernetes intenta reiniciar el pod automáticamente, pero aplica una política de backoff exponencial: espera 10 segundos, luego 20, 40, y así sucesivamente hasta un máximo de 5 minutos entre reintentos. Si el pod continúa fallando, entra en estado CrashLoopBackOff.

Este estado indica que el contenedor arranca, falla, Kubernetes lo reinicia, vuelve a fallar, y así en bucle. El pod permanece en el clúster (visible con `kubectl get pods`) pero no procesa tráfico.

Para diagnosticarlo se utilizan:

```
kubectl logs <pod>
```

```
kubectl describe pod <pod>
```

6. ¿Cómo se logra la escalabilidad en Kubernetes?

Kubernetes permite escalar horizontalmente un Deployment cambiando el número de réplicas:

```
kubectl scale deployment backend --replicas=3 -n devops-lab
```

Esto crea 3 pods del backend corriendo en paralelo. El Service del backend distribuye el tráfico entre los 3 pods automáticamente, actuando como load balancer interno. Si uno falla, los otros dos siguen atendiendo requests mientras Kubernetes recrea el tercero.

Para escalar automáticamente según la carga, Kubernetes ofrece el Horizontal Pod Autoscaler (HPA), que ajusta las réplicas dinámicamente en función de métricas como uso de CPU o memoria, sin intervención manual.

7. ¿Qué ventajas ofrece Kubernetes frente a Docker standalone?

Aspecto: Alta disponibilidad

- Docker standalone: No — si el contenedor muere, muere
- Kubernetes: Sí — recrea pods automáticamente

Aspecto: Escalado

- Docker standalone: Manual, reinicio de contenedores
- Kubernetes: Horizontal con un comando o automático (HPA)

Aspecto: Balanceo de carga

- Docker standalone: Requiere configuración extra (nginx, traefik)
- Kubernetes: Integrado nativamente en los Services

Aspecto: Gestión de configuración

- Docker standalone: Variables en docker-compose.yml
- Kubernetes: ConfigMap y Secret nativos con control de acceso

Aspecto: Despliegues sin downtime

- Docker standalone: Complejo de implementar
- Kubernetes: Rolling updates integrados

Aspecto: Almacenamiento persistente

- Docker standalone: Volumes en docker-compose
- Kubernetes: PersistentVolumeClaim gestionado por el clúster

Aspecto: Observabilidad

- Docker standalone: Logs por contenedor
- Kubernetes: Eventos, describe, métricas centralizadas

En entornos de producción, Kubernetes permite operar aplicaciones distribuidas con resiliencia, escalabilidad y facilidad de gestión que Docker standalone no puede ofrecer de forma nativa.

8. ¿Qué dificultades encontraron durante el despliegue?

La principal dificultad fue la configuración de imágenes locales en Minikube. Por defecto, Minikube usa su propio Docker daemon separado del sistema host. Al construir las imágenes con el Docker del host, Kubernetes no las encontraba y los pods fallaban con ErrImageNeverPull.

La solución fue apuntar el cliente Docker al daemon de Minikube con:
`eval $(minikube docker-env)`

Otra dificultad fue la configuración del `VITE_API_URL` en el frontend. Vite embebe las variables de entorno en tiempo de build, no en runtime. Esto requirió conocer de antemano la IP de Minikube:
`minikube ip`

Por último, el orden de despliegue resultó crítico: el backend fallaba al iniciar si PostgreSQL todavía no estaba listo para aceptar conexiones, lo que generó pods en estado `CrashLoopBackOff` durante las primeras pruebas.

9. ¿Tuvieron errores y cómo diagnosticaron los errores?

Sí. Los comandos principales que se utilizaron para diagnóstico fueron:

Ver estado de pods y detectar errores como `CrashLoopBackOff` o `ErrImageNeverPull`
`kubectl get pods -n devops-lab`

Ver eventos detallados del pod
`kubectl describe pod <nombre-pod> -n devops-lab`

Ver logs del contenedor
`kubectl logs <nombre-pod> -n devops-lab`

Ver logs en tiempo real
`kubectl logs -f deployment/backend -n devops-lab`

El flujo de diagnóstico fue:

1. `kubectl get pods` para identificar el pod problemático y su estado.
2. `kubectl describe pod` para ver eventos de Kubernetes.
3. `kubectl logs` para ver errores propios de la aplicación.

10. ¿Qué mejoras implementarían en producción?

1. CI/CD pipeline:

Automatizar el build de imágenes y el despliegue con GitHub Actions o GitLab CI.

2. Registry de imágenes privado:

Usar Docker Hub, Amazon ECR o un registry privado en lugar de imágenes locales.

3. Secrets gestionados externamente:

Reemplazar los Secrets de Kubernetes con soluciones como HashiCorp Vault o AWS Secrets Manager.

4. Ingress Controller con TLS:

Reemplazar los NodePort por un Ingress Controller con HTTPS.

5. Horizontal Pod Autoscaler (HPA):

Configurar escalado automático basado en métricas de CPU/memoria.

6. Health probes HTTP:

Cambiar las probes TCP por endpoints HTTP reales como:

/api/health

7. Resource requests y limits:

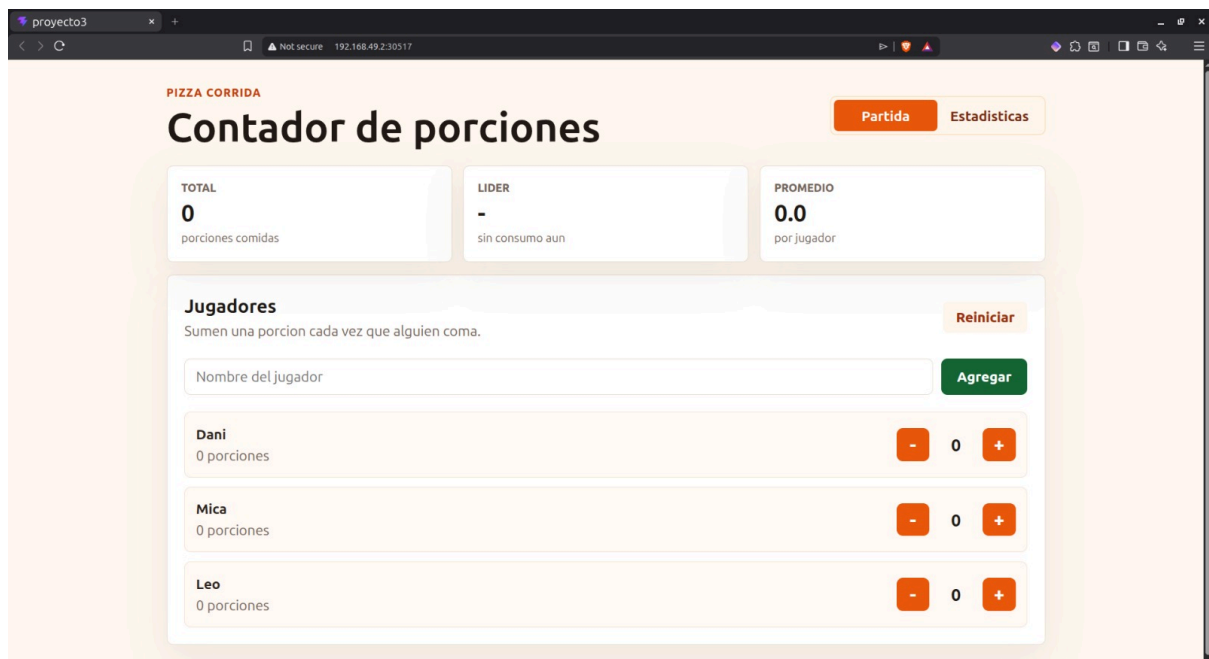
Definir límites de CPU y memoria en cada Deployment.

8. Múltiples réplicas desde el inicio:

Configurar al menos 2 réplicas del backend para garantizar disponibilidad.

Evidencias

```
jorge@jorge-Nitro-ANV15:~/Documents/DevOps/proyecto-grupal-kubernetes$ kubectl get pods -n devops-lab
NAME                                READY   STATUS    RESTARTS   AGE
backend-78ff8f6d69-f99sg            1/1     Running   4 (4m10s ago)    6h54m
frontend-684bfbbd7-5qbwk           1/1     Running   2 (5h23m ago)    7h19m
postgres-679d74b98c-q8fp8          1/1     Running   2 (5h23m ago)    7h30m
jorge@jorge-Nitro-ANV15:~/Documents/DevOps/proyecto-grupal-kubernetes$ kubectl get services -n devops-lab
NAME                TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
backend-service     NodePort    10.98.18.245  <none>         3000:30300/TCP   7h30m
frontend-service    NodePort    10.102.50.176 <none>         5173:30517/TCP   7h30m
postgres-service    ClusterIP   10.106.136.164 <none>         5432/TCP          7h30m
jorge@jorge-Nitro-ANV15:~/Documents/DevOps/proyecto-grupal-kubernetes$ kubectl get pods -n devops-lab
NAME                                READY   STATUS    RESTARTS   AGE
backend-78ff8f6d69-f99sg            1/1     Running   4 (6m5s ago)    6h56m
backend-78ff8f6d69-n9p67            1/1     Running   0           39s
backend-78ff8f6d69-rtrtz            1/1     Running   0           39s
frontend-684bfbbd7-5qbwk           1/1     Running   2 (5h25m ago)    7h21m
postgres-679d74b98c-q8fp8          1/1     Running   2 (5h25m ago)    7h32m
jorge@jorge-Nitro-ANV15:~/Documents/DevOps/proyecto-grupal-kubernetes$ kubectl delete pod backend-78ff8f6d69-f99sg -n devops-lab
pod "backend-78ff8f6d69-f99sg" deleted from devops-lab namespace
jorge@jorge-Nitro-ANV15:~/Documents/DevOps/proyecto-grupal-kubernetes$ kubectl get pods -n devops-lab -w
NAME                                READY   STATUS    RESTARTS   AGE
backend-78ff8f6d69-n9p67            1/1     Running   0           92s
backend-78ff8f6d69-pmkdk            0/1     Running   0           16s
backend-78ff8f6d69-rtrtz            1/1     Running   0           92s
frontend-684bfbbd7-5qbwk           1/1     Running   2 (5h26m ago)    7h22m
postgres-679d74b98c-q8fp8          1/1     Running   2 (5h26m ago)    7h33m
```



6. Conclusiones

Este proyecto permitió experimentar en la práctica los conceptos fundamentales de Kubernetes. La diferencia más significativa respecto a Docker Compose fue la capacidad

del clúster de mantener el estado deseado de forma autónoma: pods que se recrean solos, servicios que permanecen estables ante cambios en los pods, y escalado horizontal con un solo comando.

La curva de aprendizaje principal estuvo en entender la separación entre el entorno del host y el entorno de Minikube, y en comprender que Kubernetes no solo ejecuta contenedores, sino que los opera de forma continua garantizando disponibilidad y consistencia.